

ZyncSwap Assessment

A Beginner-Friendly Guide

No trading knowledge needed

What the project is • What the client wants

What is already built • How to build the rest, step by step

Prepared for: Adil Tariq

Date: 11 September 2026

Contents

Part 1 — What is this project?

Part 2 — What is its main purpose?

Part 3 — Words you must know (in plain English)

Part 4 — What is ALREADY built (you get this free)

Part 5 — What the client WANTS (the 3 requirements)

Part 6 — How to solve it, in very small steps

Part 7 — Map of the files (where to look)

Part 8 — Your work plan, hour by hour

Part 9 — The README you must write

Part 10 — How to check your work before submitting

Read this first. You do not need to know anything about trading or crypto. This whole project is really just: **a list of items stored in the computer's memory, a screen that shows that list, and the rules for adding and removing items from it.** That is all. Every difficult word in the task description is just a fancy name for something very simple. Part 3 translates them all.

Part 1 — What is this project?

The project is called **ZyncSwap**. It is a **website** where a person can watch the prices of different coins going up and down, and can place **pretend orders** to buy or sell them.

Nothing here is real. No real money. No real coins. No bank. No blockchain. It is a practice / demo app. When you "buy" something, the app just writes a line into a file on the computer. That is it.

A simple way to picture it

Imagine an online shop that sells only one thing: numbers.

- The **price list** keeps changing every 2 seconds, because a real price website (Binance) is sending the app fresh prices all day.
- A customer can say: *"I want to buy 5 units, but only if the price drops to 100."* The shop writes that request on a sticky note and puts it on the wall.
- Another customer can say: *"I want to sell 5 units at 100."*
- When a buy note and a sell note agree on the price, the shop **staples them together** and both requests are done.

That wall of sticky notes has a fancy name: the **order book**. The staff member who matches the notes has a fancy name: the **matching engine**. That is honestly the whole product.

What kind of code is it?

It is **one single Next.js application** that does both jobs at once:

Part	What it means in plain words	Where it lives
Frontend	The screens the user sees and clicks. Written in React + TypeScript.	<code>client/src/</code> and <code>client/app/(dex)/</code>
Backend	The brain. Keeps the list of orders, does the matching, talks to Binance.	<code>client/lib/</code> and <code>client/app/api/</code>
Server file	Starts everything up and opens the live connection.	<code>client/server.js</code>

Important rule from the client: you only work inside the `client/` folder. The `contracts/` folder is blockchain code — **ignore it completely**. The task says clearly: no blockchain, no wallet, no smart-contract work is required.

Part 2 — What is its main purpose?

Here is the honest answer, and it is important that you understand it:

The purpose of this project is NOT to build a trading website. The trading website is already built and working.

The purpose is to test whether you can add three "professional-grade" improvements to software that already exists — without breaking it.

It is a **job assessment**. A company gave you a working app and said: "Here are three hard problems. Fix them. We will judge how you think."

What are they actually testing?

They want to see if you can...	Meaning in plain words
Handle a broken internet connection	If the user's WiFi drops for 10 seconds and comes back, does the screen still show correct information? Or does it show old, wrong data?
Handle two things happening at the same moment	If two requests hit the server at the exact same millisecond, does the data get corrupted?
Handle a double-click	If a user clicks "Submit" twice by accident, do you create two orders instead of one?
Think about safety	If the price information is old or wrong, do you still let the user act on it? (You should not.)
Explain your decisions	Can you write down <i>why</i> you chose your approach? This is graded.

Scoring weight: Requirement 1 = **40%**, Requirement 2 = **35%**, Requirement 3 = **25%**.

Time budget: 4 to 6 hours. The client says clearly — a small amount of *correct, well-explained* work beats a large amount of sloppy work.

The biggest danger: the app already works. If your changes break something that used to work (the dashboard, the markets page, the trade page), you lose marks **harshly**. Test the old pages after every change.

Part 3 — Words you must know (in plain English)

Every scary word in the task description is explained below. Read this page twice and the requirements will suddenly make sense.

Trading words

Scary word	What it really means
Market	A pair of coins, e.g. <code>btc-usdt</code> . Think of it as a category ID. The app has 35 of them.
Order	A single request from the user: "buy 5" or "sell 2". It is one row of data with an ID, a size, a price and a status.
Limit order	"Buy 5, but only at price 100 or cheaper." It waits on the wall until someone agrees. It can sit there for a long time.
Market order	"Buy 5 right now, I don't care about the price." It executes immediately against whatever is on the wall. No waiting.
Order book	The wall of waiting limit orders. Two lists: people wanting to buy (bids) and people wanting to sell (asks).
Open order	An order still waiting on the wall. Nothing has happened to it yet.
Fill / Trade	When a buy and a sell are matched, that is a "fill". The order is done.
Partial fill	You wanted 10, but only 4 were available. So 4 done, 6 still waiting.
Cancel	User removes their waiting order from the wall. It never happens.
Matching engine	The code that compares buy orders with sell orders and pairs them up. File: <code>lib/matchingEngine.js</code> .
Mark price	The current "official" price of a coin, taken from Binance. Just a number that keeps changing.
Feed	The stream of prices arriving from Binance. "Feed is down" = we stopped receiving fresh prices.
Notional exposure	Sounds terrifying. It is one multiplication: <code>how much is still waiting × current price</code> , added up for all orders. It answers: "how much money is on the line right now?"

Technical words

Scary word	What it really means
Polling	The browser asking the server " <i>anything new? anything new? anything new?</i> " every few seconds. Wasteful, and slow to notice changes. <i>This app does it right now — see <code>TradePage.tsx</code>, it calls the server every 4 seconds.</i>
WebSocket	A phone line that stays open between browser and server. The server can now speak first instead of waiting to be asked. Much faster, much less waste. <i>This app already has one at <code>/ws/markets</code> for prices.</i>
Push-based	The server sends you news the moment it happens (WebSocket). The opposite of polling.
Snapshot	"Here is the complete current picture." Sent once when you connect, so you start from a correct state.
Incremental event	"Here is just the one small thing that changed." Sent after the snapshot. Much smaller than sending everything again.
Sequence number	A counter you attach to every message: 1, 2, 3, 4... If the browser receives 1, 2, 3, then 5 — it knows message 4 was lost. That is a gap . Then it can ask for a fresh snapshot.
Monotonic	Only goes up, never back down. (1, 2, 3, 4 — never 3, 2, 5.)
Re-hydrate	After reconnecting, throw away what you had and reload the full correct picture.
Idempotency	The hardest-sounding word, the simplest idea: "doing it twice has the same result as doing it once." The user clicks Submit twice → only one order is created. You achieve this by making the browser send a unique ticket number (an idempotency key) with the request. If the server has seen that ticket before, it returns the old answer instead of doing the work again.
Atomic	All-or-nothing, and nobody can interrupt halfway.
Concurrency	Two or more things happening at the same time, fighting over the same data.
Mutex / lock / critical section	A "one person at a time" rule. Like a single toilet with a lock on the door — everyone else waits in line. Prevents two operations from corrupting the data.
Circuit breaker	Named after the electrical switch in your house. When something is unsafe, it automatically switches off the dangerous feature, and switches it back on when things are safe again.
Stale	Old, out of date. "The price is stale" = we haven't had a fresh price in a while, so we cannot trust it.
In-memory	Stored in RAM, not in a database. Disappears if the server restarts — which is why this app also writes it to a JSON file as a backup.

Part 4 — What is ALREADY built

You did not start from zero. Here is what the client already gave you, working.

Working pages

Page	What it shows
<code>/dashboard</code>	A summary: how many markets, total volume, biggest gainers and losers, how many orders are open.
<code>/markets</code>	A table of all 35 coin pairs with live changing prices, plus a badge showing if the connection is alive.
<code>/trade/btc-usdt</code>	The main screen: a price chart, the order book, a form to place an order, and a list of your open orders.

Working API endpoints (the backend addresses)

Address	What it does
<code>GET /api/health</code>	"Are you alive?"
<code>GET /api/v1/config</code>	App name and settings.
<code>GET /api/v1/dashboard/summary</code>	The numbers for the dashboard page.
<code>GET /api/v1/markets</code>	All 35 markets with prices.
<code>GET /api/v1/markets/:id</code>	One market in detail.
<code>GET /api/v1/orders</code>	List the orders.
<code>POST /api/v1/orders</code>	Create a new order.
<code>DELETE /api/v1/orders/:id</code>	Cancel one order.
<code>WS /ws/markets</code>	The open phone line that pushes price updates only .

How data moves today

PRICES (already push-based – good)

Binance → marketEngine.js → WebSocket `/ws/markets` → Browser

ORDERS (still polling – this is the problem)

Browser → `POST /api/v1/orders` → `matchingEngine.js` → JSON file

Browser asks again every 4 seconds ↺ `GET /api/v1/orders`

See the difference? Prices already travel on the fast open phone line. Orders do not — the browser keeps re-asking. **Requirement 1 is simply: put orders on the phone line too.**

What is missing (your job)

- Orders do not stream live — the page polls every 4 seconds.
- Double-clicking Submit creates two orders. Nothing prevents it.
- There is no "Cancel all" button.
- Two requests at the same time can corrupt the order list. There is no lock.
- If the price feed dies, the app still happily accepts orders at unknown prices.
- The dashboard only shows a simple true/false for the connection. No detail.
- There are only 7 basic tests.

Part 5 — What the client WANTS

Requirement 1 — Live order updates 40%

The problem in one sentence

When you place an order, the screen does not update immediately — you wait up to 4 seconds, because the browser is on a "ask again and again" schedule.

What the client wants

The server should **immediately tell the browser** when anything happens to an order, using the open phone line (WebSocket). And it must survive a dropped connection.

What you must deliver

1. **A message format (a "protocol")**. You decide what the messages look like. You need at least: one **snapshot** message (the full current list, sent on connect), and small **event** messages (order created / partly filled / fully filled / cancelled). Every message carries a counter number.
2. **Server side**. The messages must be sent from the *place where the order actually changes* — inside the matching engine. Not by re-reading the API.
3. **Browser side**. The trade page subscribes to the stream. If the connection drops, show a "Reconnecting..." message on screen, and when it comes back, reload the full correct list so the user never sees wrong data.
4. **Write it down** in the README: your message format, how you detect a lost message, and why you chose one WebSocket instead of two.

What loses marks: just sending the same old API response down the WebSocket with no snapshot and no counter. They call this "a thin websocket echo" and say it will not score well.

Requirement 2 — Make the order engine safe 35%

The problem in one sentence

The order list lives in memory with no protection. A double-click makes two orders, and two simultaneous requests can leave the data in a broken state.

Three things to build

(a) Stop duplicate orders.

The browser generates a random ticket number and sends it with the order request. The server keeps a note of ticket numbers it has already handled. If the same ticket arrives again, the server does **not** create a second order — it just returns the first answer again.

(b) A "Cancel all" feature.

Add a new address `POST /api/v1/orders/cancel-all` that cancels every waiting order for one market **in one single protected operation** — so no new order can sneak in halfway through. It must return a clean summary like `{ "cancelled_count": 4, "ids": [...] }`. Then add a **Cancel all** button on the trade page, with a "Are you sure?" confirmation.

(c) A queue so only one thing runs at a time.

Submit, cancel and cancel-all must all pass through the same "one at a time" door. This is the single most important idea in this requirement.

Tests you must write

- Sending the same order twice with the same ticket creates only one order.
- Cancel-all really clears several open orders.
- A test that fires a submit and a cancel-all at the same time and checks that the result still makes sense.

What loses marks: writing cancel-all as a simple `for` loop that calls `cancel()` over and over with no shared lock. The client names this exact mistake in the task description.

Requirement 3 — The safety switch 25%

The problem in one sentence

If the connection to Binance dies, the app keeps showing the last known price forever, and still lets the user place instant orders based on that old, untrustworthy number.

What you must build

(a) Tell the truth about the feed.

Right now the dashboard only says `feed_connected: true/false`. Add more honest detail:

- `feed_connected` — is the line to Binance open?
- `feed_mode` — `"live"` (real prices) or `"simulated"` (made-up prices).
- `last_tick_age_ms` — how many milliseconds since the last real price arrived.

These numbers must come from the real engine, not from a fake timer you invent.

(b) The switch itself.

You pick a limit — for example, *"if no fresh price for more than 10 seconds, the feed is unsafe."* When unsafe:

- The instant-order button in the UI becomes disabled, with a visible reason.
- The API also refuses it, returning a plain text error with status `503` or `409` — you choose one and explain why.

Both layers are required. The client says a "feed down" badge that still lets the API accept the order "will not score well". The UI check is politeness; the API check is the real protection.

You may allow waiting-type (limit) orders to continue, as long as you explain your reasoning. Or block everything. Either is accepted if defended.

(c) An operator panel on the dashboard showing: feed health, whether the switch is on or off, how many orders are open, and the total money at risk (`remaining size × current price`, summed).

(d) At least one test proving that an unsafe feed causes an instant order to be rejected.

Part 6 — How to solve it, in very small steps

Suggested order: do Requirement 2 first, then 3, then 1.

Why? Requirement 2 is pure backend logic — easiest to test and hardest to break. Requirement 3 is small. Requirement 1 is the biggest and touches both sides, so do it when you understand the code well. But note: **Requirement 1 is worth the most marks**, so do not run out of time before reaching it. If you are unsure, do a simple version of Requirement 1 early, then improve it later.

Before you write any code

1 Run it.

```
npm install
npm run dev
```

Open `http://localhost:3000`. Click through `/dashboard`, `/markets`, `/trade/btc-usdt`. Get a feel for it.

2 Place an order by hand. On the trade page, place a "wait until price X" order with a price far from the current one, so it stays waiting. Watch it appear in the "Open orders" tab. Now cancel it. Now you have seen the whole loop.

3 Run the tests.

```
npm run test
```

7 tests should pass. This is your safety net — run it after every change.

4 Read two files only. `client/server.js` (90 lines — the starting point) and `client/lib/matchingEngine.js` (the brain — focus on the `submit()` and `cancel()` functions).

Requirement 2 — step by step

1 Build the "one at a time" door. In `lib/matchingEngine.js`, add a tiny helper to the class. The idea in plain code:

```
// a chain of promises – each job waits for the previous one
#queue = Promise.resolve();

#runExclusive(job) {
  const result = this.#queue.then(job, job);
  // keep the chain alive even if a job throws
  this.#queue = result.then(() => {}, () => {});
  return result;
}
```

Now wrap the **body** of `submit()` and `cancel()` inside `#runExclusive(...)`. Nothing else changes yet. Run `npm run test` — the 7 tests must still pass.

2 Add the ticket system (idempotency). Give `submit()` one extra argument: `idempotencyKey`. Inside the protected section, before doing any work:

- If the key is already in your `Map`, return the stored answer. Stop.
- Otherwise do the work, then store the answer under that key.

Also store a timestamp with each entry, and delete entries older than (say) 10 minutes. Write that choice down for the README.

3 Read the key in the API route. In `app/api/v1/orders/route.js`, read the header `Idempotency-Key` (or a `idempotency_key` field in the body) and pass it through to `submit()`.

4 Send the key from the browser. In `src/components/dex/TradeEntryPanel.tsx`, generate `crypto.randomUUID()` **once** before sending, and include it as a header. Important: if the request fails and you retry, reuse the **same** key — that is the whole point.

5 Write `cancelAll()`. Add a method to the engine that, inside **one single** protected section, walks the list of open orders for a market and cancels them all, then saves once at the end. Return `{ cancelled_count, ids }`.

Do not call your existing `cancel()` in a loop — that would take and release the lock many times, which is exactly the mistake the client warns about. Extract the cancel logic into a small private un-locked helper and call that.

6 Create the new API route. New file: `app/api/v1/orders/cancel-all/route.js`. It reads `market_id`, returns plain-text `400` if it is missing, plain-text `404` if the market does not exist, and JSON on success. Never let a crash message leak out — wrap it in try/catch.

7 Add the button. In `src/views/TradePage.tsx`, in the "Open orders" tab, add a **Cancel all** button. On click, show a confirm dialog, then call the new address, then refresh the list.

8 Write the tests in `client/test/`. Three of them, as listed in Part 5. For the concurrency one, fire the calls together with `Promise.all([...])` and then assert the end state is sensible (for example: no order is both cancelled and still sitting in the book).

Requirement 3 — step by step

1 Record the time of the last price. In `lib/marketEngine.js`, find where a message from Binance is handled. Add one line: `this._lastTickAt = Date.now();`

2 Add a health function. Still in `marketEngine.js`, add:

```
feedHealth() {
  const connected = this.isFeedConnected();
  const ageMs = this._lastTickAt ? Date.now() - this._lastTickAt : null;
  return {
    feed_connected: connected,
    feed_mode: connected ? "live" : "simulated",
    last_tick_age_ms: ageMs,
  };
}
```

3 Make a tiny rules file. Create `lib/feedPolicy.js` with one pure function — no imports, no state, which makes it trivially testable:

```
export const STALE_MS = 10_000;

export function isFeedSafe(health) {
  if (!health.feed_connected) return false;
  if (health.feed_mode !== "live") return false;
  if (health.last_tick_age_ms == null) return false;
  return health.last_tick_age_ms <= STALE_MS;
}
```

4 Block it in the API. In `app/api/v1/orders/route.js`, inside `POST`, before submitting: if the order type is instant (`market`) and `isFeedSafe()` is false, return a plain-text message with status `503`. Done.

5 Expose it to the frontend. In `lib/dashboardSummary.js`, spread the health object into the result, and add `breaker_open` plus `open_notional`. Calculate exposure like this:

```
let open_notional = 0;
for (const o of openOrders) {
  const m = markets.find((x) => x.id === o.market_id);
  const mark = m?.mark_price ?? 0;
  const px = o.order_type === "limit" ? (o.price ?? mark) : mark;
  open_notional += o.size_remaining * px;
}
```

Write the formula and its limitations in the README (for example: it ignores leverage, and uses the last known price which may itself be stale).

6 Show it on screen. In `src/views/DashboardPage.tsx`, add a small panel: feed status, mode, seconds since last price, breaker ON/OFF, open order count, and total exposure.

7 Disable the button. In `TradeEntryPanel1.tsx`, fetch the summary (or read it from the existing stream) and grey out the instant-order button when the feed is unsafe, with a short explanation next to it.

8 One test. Call `isFeedSafe()` with a stale health object and assert it returns `false`. Because you made it a pure function, this test is three lines.

Requirement 1 — step by step

Decision to make first: one WebSocket or two?

Recommendation: add a second path, `/ws/trading`. Reason — prices go to everybody and change constantly; order news is specific to a market and much rarer. Keeping them separate means one can be reconnected without disturbing the other. **Write this reason in the README** — they explicitly ask for it.

1 Make the engine an announcer. In `lib/matchingEngine.js`, make the class `extends EventEmitter` (import from `node:events`). Add a counter field starting at 0.

2 Announce every change. Add a small private helper:

```
#emitEvent(type, payload) {
  this.seq += 1;
  this.emit("trading", {
    v: 1,
    seq: this.seq,
    type,                      // order_accepted | order_partially_filled
                                // | order_filled | order_cancelled
    market_id: payload.market_id,
    ts: Date.now(),
    data: payload,
  });
}
```

Call it at the end of `submit()`, inside `cancel()`, for each order affected by `cancelAll()`, and for each maker order touched during matching.

3 Add a snapshot function.

```
async snapshotFor(marketId) {
  return {
    v: 1,
    type: "snapshot",
    seq: this.seq,              // the counter at this exact moment
    market_id: marketId,
    orders: await this.listOrders(marketId, true),
    recent_fills: await this.recentTrades(marketId, 50),
  };
}
```

This is the most important piece. It is what makes reconnecting safe.

4 Open the new path in `server.js`. Create a second `WebSocketServer`, and in the existing `upgrade` handler add a branch for `/ws/trading` (the file already has an `if/else` there — just add a second `if`).

5 Handle a client connecting. When a browser connects it sends `{ "op": "subscribe", "market_id": "btc-usdt" }`. The server then:

- sends the snapshot for that market,
- and from then on forwards every `"trading"` event whose `market_id` matches.

Remember to remove the listener on `close` — the existing `/ws/markets` code shows you exactly how.

6 Keep a small history buffer. Keep the last ~500 events in an array in the engine. If a reconnecting client says "my last counter was 42", you can replay 43 onwards. If 42 is too old to be in the buffer, just send a fresh snapshot instead. Simple, and it satisfies the "recover without silently dropping fills" requirement.

7 Write the browser side. Create `src/context/TradingStreamContext.tsx`. Honestly, the easiest path: **copy** `MarketsStreamContext.tsx` — it already has connect, auto-reconnect and status handling written for you — then change the URL and the message handling.

Your handler does three things: store the snapshot; apply each event to the list; and if `msg.seq !== lastSeq + 1`, note a gap and ask for a fresh snapshot.

8 Use it in the trade page. In `TradePage.tsx`, replace `serverOrders` with the list coming from your new context. Remove the 4-second polling loop (the `useEffect` containing `setInterval(refreshOrders, 4000)`).

Safer option: keep a slow safety poll at 30 seconds and say in the README that you kept it deliberately as a backup. That is an accepted answer.

9 Show the connection state. Add a small badge near the Open orders tab: **Live** / **Reconnecting...** / **Offline**. The client explicitly asks for this to be visible.

10 Test it by hand. Open the trade page, then stop the server (`Ctrl+C`), watch the badge change to "Reconnecting...", start the server again, and confirm the order list comes back correct.

Part 7 — Map of the files

Read them in this order. The line counts tell you how big each one is.

Backend (the brain) — read first

File	Lines	What it does	You touch it?
<code>client/server.js</code>	90	Starts everything. Opens <code>/ws/markets</code> . Your entry point.	Yes — Req 1
<code>client/lib/engines.js</code>	17	Just hands you the two engines. Tiny.	No
<code>client/lib/matchingEngine.js</code>	544	The most important file. Holds all orders, matches buys with sells.	Yes — Req 1 & 2
<code>client/lib/marketEngine.js</code>	280	Gets prices from Binance, broadcasts every 2 seconds.	Yes — Req 3
<code>client/lib/dashboardSummary.js</code>	42	Builds the dashboard numbers. Small and easy.	Yes — Req 3
<code>client/lib/merge.js</code>	109	Number formatting helpers.	No
<code>client/app/api/v1/orders/route.js</code>	~75	Handles create + list orders. Good example of how routes call the engine.	Yes — Req 2 & 3
<code>client/app/api/v1/orders/[id]/route.js</code>	small	Handles cancelling one order.	Maybe
<code>client/test/lib.test.js</code>	109	The 7 existing tests. Copy their style.	Yes — all reqs

Frontend (the screens) — read second

File	Lines	What it does	You touch it?
<code>src/context/MarketsStreamContext.tsx</code>	101	The browser side of the price WebSocket. Copy this as your template.	Template
<code>src/views/TradePage.tsx</code>	311	The trade screen. Contains the polling code you will remove.	Yes — Req 1 & 2
<code>src/views/DashboardPage.tsx</code>	255	The dashboard. Your safety panel goes here.	Yes — Req 3
<code>src/components/dex/TradeEntryPanel.tsx</code>	—	The order form. Sends the create request.	Yes — Req 2 & 3
<code>src/views/MarketsPage.tsx</code>	397	The markets table. Works fine.	No
<code>src/views/HomePage.tsx</code>	1653	Marketing landing page. Huge. Ignore it completely.	No

Ignore these entirely: the `contracts/` folder, `src/wallet/`, `src/components/SwapPanel.tsx`, `MintPanel.tsx`, `lib/swapQuote.js`, `lib/zyncBalance.js`, `lib/ethCall.js`. These are all blockchain features and are **not part of the assessment**. Do not waste a single minute on them.

Part 8 — Your work plan

The client budgets 4 to 6 hours. Here is how to spend it.

Time	What to do
0:00 – 0:30	Install, run, click through all three pages. Place an order and cancel it by hand. Run the tests. Read <code>server.js</code> and the <code>submit()</code> function.
0:30 – 2:00	Requirement 2. Build the lock, the ticket system, cancel-all, the API route, the button, and the three tests.
2:00 – 2:45	Requirement 3. Feed health, the rules file, the API block, the dashboard panel, one test. This is the quickest one — get the marks.
2:45 – 5:00	Requirement 1. Events from the engine, the new WebSocket path, snapshots, the browser context, remove the polling, add the status badge.
5:00 – 5:45	Write the README. Do not skip this — it is directly graded.
5:45 – 6:00	Final check (Part 10). Commit. Push. Send the link.

If you start running out of time: stop coding and write the README instead. A half-finished feature with a clear written explanation of the design and what is missing scores better than a finished feature with no explanation. The client says this in their own words: *"Prefer a correct, well-reasoned partial solution over a shallow full one."*

Part 9 — The README you must write

You must update `readme.md` in your fork. Answer these questions. Short and clear beats long and vague.

1. How to run it

Install, run, test. Three commands. Easy marks.

2. Your message format (Requirement 1)

- Show an example snapshot message and an example event message.
- How does a browser subscribe?
- How do you notice a missing message, and what do you do about it?
- One WebSocket or two — and **why**?

- Did you keep a backup poll? Why?

3. Your safety model (Requirement 2)

- How do you make sure only one operation runs at a time?
- How long do you remember ticket numbers, and where? What happens on restart?
- State the rule you guarantee. For example: *"An order can never be both cancelled and still present in the order book."*

4. Your safety switch (Requirement 3)

- What counts as "unsafe"? What exact number of seconds did you pick, and why?
- Did you return 503 or 409 ? Why?
- Do you block waiting-type orders too, or only instant ones? Why?
- Your exposure formula, and what it does **not** account for.

5. What you would do with more time

Be specific. "Add more tests" is weak. Something like *"The ticket map lives in memory, so a server restart loses it — with more time I would move it to the same JSON file the orders use, with a cleanup pass on boot"* is strong. This section shows self-awareness, which is exactly what they are measuring.

Part 10 — Final check before submitting

Nothing old is broken

- `/dashboard` still loads and shows numbers.
- `/markets` still shows the table with prices moving.
- `/trade/btc-usdt` still shows the chart and the order book.
- Placing an order still works.
- Cancelling one order still works.
- All the original 7 tests still pass.

Requirement 1 works

- Place an order → it appears in the list **instantly**, not after 4 seconds.
- Cancel an order → it disappears instantly.
- Stop the server → the badge says "Reconnecting...".
- Start the server again → the list reloads and is correct. No stuck rows.

Requirement 2 works

- Sending the same order twice with the same ticket number creates only one order.
- "Cancel all" asks for confirmation, then clears every waiting order.
- Sending bad input gives a clean `400` message, not a crash dump.
- Your new tests pass.

Requirement 3 works

- The dashboard shows feed status, mode, age, and exposure.
- Turn off your internet → the instant-order button becomes disabled.
- Trying it through the API anyway gives a clear error, not a crash.
- Turn the internet back on → everything becomes available again by itself.

Submission

- The README is updated with all five sections from Part 9.
- **Do not open a pull request** to the original repository — the task forbids it.
- Share the URL of **your own fork**.
- Test it from scratch one last time:

```
npm install
npm run test
npm run dev
```

One last thought. You do not need to be a trading expert to do this well. Every single requirement is a normal software problem dressed up in trading clothes:

- Requirement 1 is "**push updates instead of polling, and survive a reconnect.**"
- Requirement 2 is "**don't let a double-click or a race condition corrupt my data.**"
- Requirement 3 is "**turn off the dangerous button when the data is old.**"

You have solved problems like these before. The coins are just decoration. Build it small, test after every step, and write down your reasoning.